

On this page

Historic Events

As of version 1.2.0, the system supports receiving historic events for Digital Activity and Fraud Tagging. These events have a workflow field `is_historic` set to true and are dropped during the workflow, not contributing to realtime metrics. Furthermore, historic events are inserted in the database with the `event_occurred_at` as the `FDZ_timestamp` value, instead of the Pulse clock.



In this page, please have in consideration that Transfers applies to both Inbound, Outbound Payments. Cards also follows the same logic as Transfers

How are events classified as historic?

Historic events are determined by a set of *slack* properties and the following picture describes their interaction with the different event types.

What are the current slack values?

The values chosen for the past, historic and future slacks influence how events are processed and which events are accepted by the system. Currently, they following values are in use:

	Past Slack	Historic Slack	Future Slack
Transfers	30 min	30 min	30 min
Cards	30 min	30 min	30 min
Digital Activity	30 min	30 min	30 min

Taking those values into consideration, we can affirm that the system will:

- **reject** any request (regardless of the channel) with timestamp more than **30 minutes in the future**
- **not reject** any **Digital Activity** request in the past
- treat any Digital Activity or Fraud Feedback request as **historic** if it is more than **30 minutes in the past**



What happens to Rejected events?

When an event is Rejected, the API will return an a Bad Request status code and the event will **not reach the workflow**. These events won't be processed in any way, nor stored in any database.

How are historic events handled by the ETL extractions?

The ETL extraction is a process that extracts Event Storage events and runs every hour, 10 minutes after the hour (i.e. at 00:10am, 01:10am, etc.). When the ETL runs, all events stored in the Event Storage with `FDZ_timestamp` for the previous hour are extracted, regardless of whether they are historic or not.

As mentioned above, historic events differ from regular events in the fact that they are stored in the Event Storage with the `event_occurred_at` as the `FDZ_timestamp` value. As the current configuration for historic slack is just 30 minutes, some historic events may appear in the extractions while other may fall outside the extraction window. The following examples illustrate both scenarios.

Example of an historic event not caught by the ETL extraction

Consider the current time is 15:45. In this scenario, historic events must have an `event_occurred_at` value before 15:15. The next ETL execution will be at 16:10 and extract events with `FDZ_timestamp` between 15:00 and 16:00.

An event received now (15:45) with `event_occurred_at` set to 14:30 will be considered historic and stored in the database with `FDZ_timestamp` equal to 14:30. It falls outside the next ETL extraction interval and therefore will not be extracted.

Example of an historic event caught by the ETL extraction

Keeping the same invariants as the previous example, consider the current time is 15:45, meaning historic events must have an `event_occurred_at` value before 15:15, and the next ETL execution will be at 16:10 and extract events with `FDZ_timestamp` between 15:00 and 16:00.

An event received now (15:45) with `event_occurred_at` set to 15:10 will be considered historic as it falls before the historic slack. Being historic, that event will be stored with `FDZ_timestamp` equal to its `event_occurred_at` (15:10). Unlike the previous example, despite being an historic event, the timestamp still falls inside the next ETL extraction window, and the event will be extracted.